# Replanning Against a Saboteur: Incremental Search When the Opponent Blocks the Route

V Aravindhan, Sathyanarayanan, Sahesh, Nihara
School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, India
Mentor: Dr. K. Sripriyan, Assistant Professor (Sl. Gr.), School of Artificial Intelligence

***Abstract*—Game characters that walk through a changing world must replan, and incremental search algorithms such as D* Lite promise to do so more cheaply than searching again from scratch. Most evaluations change the map at random. We ask what happens when the opponent changes it on purpose: a *saboteur* that blocks the cell on the character's planned path that lengthens its trip the most. We port the search code of our grid game, *Outwit*, to Python, verify it against the game's own test numbers, and compare BFS and A* that replan from scratch, a *lazy* A* that replans only when its path is broken, and D* Lite, on 360 problems from 24 real game maps of the Moving AI benchmarks (5760 runs). Four findings emerge. (1) Against random blocks, lazy A* has the lowest total search effort: D* Lite's first plan costs a median 1.84× A*'s, and the few replans that random blocks cause never pay this back. (2) Against the saboteur, D* Lite needs about 4× fewer expansions than both A* variants (86,989 vs. 354,770 on average; fewer in 305 of 360 problems), because every saboteur block hits the planned path and forces lazy A* to replan every time. (3) The worst single replan, the "hitch" a player would notice, is not reliably reduced: the paired test favours D* Lite only weakly overall (Holm $p = 0.042$) and not at all on two of the three map sets. (4) In our Python implementation, a D* Lite expansion costs 3–4× as long as an A* expansion, which cancels its expansion savings in wall-clock time.**

***Index Terms*—heuristic search, incremental search, D* Lite, A*, replanning, game AI, network interdiction**

## I. Introduction

A non-player character (NPC) in a video game often has to walk from one place to another while the world changes around it: doors close, crates fall, and, in many games, the player deliberately puts obstacles in its way. Each change can make the character's current route invalid or suboptimal, and the character must then *replan*. Game AI usually does so in the simplest way, by running A* [1] again from the character's current position [2], [3]. Incremental heuristic search algorithms [4] such as Focussed D* [5], Lifelong Planning A* [6] and D* Lite [7], [8] were designed for this situation: they keep the results of the previous search and repair only the part that the change has made wrong. For robots that discover obstacles gradually, they are known to save a large amount of work [8].

Whether they also save work in a game is less clear. Hernández et al. [9] pointed out that it is not well understood when D* Lite is actually faster than repeated A*, and later showed that a carefully designed A* variant can beat it on common map types [10]. All of these evaluations share one assumption: the changes are *random* or are discovered by sensing, and nobody chooses them. In a game the opponent is often a person who *wants* the character to fail. Our own game, *Outwit*, is an example:

the player is a knight who locks doors and drops barricades to stop a thief from stealing a treasure. Choosing which cell to block so that an agent's shortest path becomes as long as possible is the classic *shortest-path network interdiction* problem from operations research [11]–[13]. Its consequences for incremental replanning have, to our knowledge, not been measured.

This paper asks one question: *when the map is changed by an opponent who deliberately blocks the character's route, does incremental replanning (D* Lite) still save search effort compared with replanning from scratch (A*, BFS), and compared with a lazy A* that replans only when its path is actually broken?* We had no expected answer and report what the data show.

Our contributions are:

- a literal Python port of the search code of our game (a binary heap, BFS, A* and D* Lite), verified against the game's own test numbers, including its tie-breaking and its count of expansions (Section IV);
- four change models, from no change to a greedy *saboteur* that blocks, among cells on the character's planned path, the one that lengthens its trip the most (Section IV-C);
- a paired experiment with 5760 runs on 360 problems from 24 real game maps of the Moving AI benchmarks [14], analysed with bootstrap confidence intervals, Wilcoxon signed-rank tests, effect sizes and a multiple-testing correction (Sections V and VI);
- an honest statement of what this means for our game (Section VII) and of the limits of the study (Section VIII).

## II. Related Work

### A. Heuristic and incremental search

A* [1] expands nodes in order of $f = g + h$ and, with an admissible heuristic, returns an optimal path; it is the standard pathfinding method in games and in AI textbooks [15]. When the graph changes, A* has to start again. Stentz's D* [16] was the first algorithm to repair a path efficiently and optimally when a robot discovers new obstacles, and Focussed D* [5] added a heuristic to focus the repairs. Ramalingam and Reps [17] gave an incremental algorithm for shortest paths (DynamicSWSF-FP) that updates only the vertices whose distances change. Lifelong Planning A* (LPA*) [6] combined this idea with A*'s heuristic, and D* Lite [7] adapted LPA* to a moving agent by searching backwards from the goal, so that the agent's movement does not invalidate the search. D* Lite behaves like Focussed D* but is simpler to understand and analyse [7]; Koenig and Likhachev [8] showed large savings over repeated A* for robots in unknown terrain, and Koenig et al. [4] give an overview of incremental heuristic search. Later variants trade optimality for speed (Anytime D* [18]) or handle a moving goal (Moving Target D* Lite [19]).

### B. When does incremental search pay?

Incremental search is not always faster. Hernández et al. [9] argued that repeated A* is often faster than D* Lite on easy navigation problems and that incremental algorithms should be tested on more varied problems. Hernández, Baier and Asín [10] proposed Multipath Adaptive A* (MPAA*), which reuses paths from earlier searches, and reported it to be faster than D* Lite on most of their test maps, but not on mazes. These studies change the map by sensing or at random. Our study keeps their question but changes who chooses the changes.

### C. Pathfinding in games

Commercial games use grid or graph maps and favour fast, simple methods such as hierarchical path-finding (HPA*) [20] and symmetry pruning (Jump Point Search) [21]. Sturtevant's benchmark set [14], which we use, made results on real game maps comparable across papers. Baier et al. [22] studied a hunter chasing a prey on such maps when part of the map is unknown, combining precomputed path databases with online incremental search. For NPC decision making, Orkin's goal-oriented planner in F.E.A.R. [2] uses A* both for paths and for action plans and replans when the world changes; Uludağlı and Oğuz [3] survey decision-making methods for NPCs and their costs.

### D. Network interdiction

Fulkerson and Harding [11] studied how to lengthen arcs within a budget so as to maximise the shortest source–sink path, and showed that the problem is equivalent to a parametric minimum-cost flow problem. Israeli and Wood [12] studied shortest-path network interdiction, in which an interdictor with a limited budget attacks arcs to maximise the length of the shortest path of an evader, as a two-level (max–min) optimisation problem and solved it with decomposition algorithms. Smith and Song [13] survey the field, including dynamic and sequential variants. Our saboteur is a deliberately simple, online version of this idea: it blocks one cell at a time, greedily, looking only one step ahead, and it does not solve the two-level problem. Its job here is not to be an optimal interdictor but to produce changes that *always* hit the character's route. Studies of interdiction optimise the attacker; we measure the replanning cost of the defender.

## III. Background

### A. Grid model

A map is a grid of cells $(x, y)$. A cell is free or blocked; cells outside the map are blocked. The character moves in four directions (right, left, down, up, in exactly this order, as in our game), and each step costs 1. The heuristic is the Manhattan distance $h(a, b) = |a_x - b_x| + |a_y - b_y|$, which never overestimates the number of steps and is therefore admissible [15]. We call one cell taken off a search's open list an *expansion*; expansions are our main measure of search effort, because they do not depend on the computer.

### B. BFS and A*

Breadth-first search expands cells in order of their distance from the start using a FIFO queue. With unit costs it finds a shortest path, but it does not use the goal's position. A* [1] expands cells in order of $f(n) = g(n) + h(n)$; our version breaks ties in $f$ toward the smaller $h$, i.e., toward cells closer to the goal, exactly as the game does.

### C. D* Lite

D* Lite [7] searches *backwards*: $g(s)$ estimates the cost from $s$ to the goal. It also keeps a one-step look-ahead value

$$rhs(s) = \min_{s' \in Succ(s)} \big(c(s, s') + g(s')\big), \tag{1}$$

with $rhs(s_{\text{goal}}) = 0$. A cell is *consistent* when $g(s) = rhs(s)$; only inconsistent cells are in the priority queue, ordered by the two-part key

$$k(s) = \big[\min(g, rhs) + h(s_{\text{start}}, s) + k_m;\ \min(g, rhs)\big], \tag{2}$$

compared lexicographically. When the agent moves from $s_{\text{last}}$ to $s_{\text{start}}$, $k_m$ grows by $h(s_{\text{last}}, s_{\text{start}})$, so that old keys stay valid lower bounds without re-sorting the queue. When a cell changes, the *rhs*

values of the cell and its neighbours are recomputed, and `compute_shortest_path` processes inconsistent cells until the agent's own cell is consistent. Because the search is rooted at the goal, a change far from the goal usually affects few $g$ values, while a change near the goal can invalidate many of them.

### D. Why lazy A* is a strong baseline

Our changes only ever *block* cells. Blocking removes vertices from the graph, so no distance can become shorter: if $G'$ is the graph after the change, then $d_{G'}(u,t) \geq d_G(u,t)$ for every $u$. If the remaining planned path $P$ from the agent $u$ to the goal $t$ uses no newly blocked cell, it still exists in $G'$ and has length $|P| = d_G(u,t) \leq d_{G'}(u,t)$, so it is still optimal. A lazy A* that replans only when a newly blocked cell lies on its remaining path therefore always follows an optimal route, and does no search at all when a block misses its path. This makes it a strong baseline against random blocks, which rarely hit the path.

## IV. Method

### A. Porting and verifying the game's search code

Outwit is written in Godot's GDScript. We ported its priority queue, BFS, A* and D* Lite line by line to Python. The priority queue is a binary heap that compares only the two key parts $(k_1, k_2)$; we deliberately did not use Python's `heapq`, which would also compare the cells and so change the order of ties. D* Lite uses lazy deletion through a dictionary of the live queue entries, as in the game. Expansions are counted at exactly the same places as in the GDScript code.

The port is accepted only if it reproduces the game's own test numbers on the game's level (`level1.txt`, start `G`, goal `T`); Table I shows that it does, exactly. A third test, ported from the game, flips three random cells per step on 40 random $20\times15$ maps with 22% walls and checks after every change that D* Lite's cost equals that of a fresh A* search; it passes. We also re-ran the tests with deliberate bugs: a different neighbour order and the key $(f, g)$ instead of $(f, h)$ both change the counts (the second gives 77 expansions instead of 67) and are caught. A different order of cells with exactly equal keys, however, does not change any count on `level1.txt`, so this level cannot prove that our tie order equals the game's (Section VIII).

TABLE I
The Python port reproduces the game's test numbers on `level1.txt`.

| Test | Game | Port |
|---|---|---|
| BFS: path cells / expansions | 28 / 190 | 28 / 190 |
| A*: path cells / expansions | 28 / 67 | 28 / 67 |
| D* Lite: expansions / $g$(start) | 75 / 27 | 75 / 27 |
| Wall drop: blocked cell | (6, 3) | (6, 3) |
| Wall drop: repaired path = fresh A* path | 24 | 24 |
| Wall drop: D* Lite repair vs. fresh A* | 3 vs. 58 | 3 vs. 58 |

### B. The four planners

All planners share one interface (`init`, `next_step`, `current_path`, `on_move`, `on_change`) and plan on the current true grid: every change is announced to the character at once (full knowledge).

- **BFS restart** runs BFS again from the agent after every change event.
- **A* restart** runs A* again from the agent after every change event.
- **A* lazy** keeps its path and runs A* again only if a newly blocked cell lies on the remaining path (Section III).
- **D* Lite** keeps one instance for the whole trip. Like the character in our game, it calls `move_start` and `compute_shortest_path` after every step, and `notify_changed` before `compute_shortest_path` after a change. The per-step calls did 0 expansions in all runs.

Only expansions inside the planners' searches are counted; the opponent's own searches are not.

### C. Change models ("pranksters")

A prankster may block one cell every $K = 5$ agent steps, at most $B = 30$ cells per run. It never blocks the agent's cell or the goal, and never a cell that would disconnect the agent from the goal (checked by search); if a drawn cell breaks a rule it tries another, at most 20 times, and otherwise skips its turn. Each prankster has its own random generator, seeded from the master seed, the map and the problem, so every planner faces the same random draws.

- **None**: never blocks (control).
- **Random global**: a uniformly random free cell anywhere on the map, the usual model in earlier studies.
- **Random local**: a uniformly random free cell within Chebyshev distance $R = 8$ of the agent, to separate "near the agent" from "on purpose".
- **Saboteur**: a greedy, one-step interdictor. It takes the character's current planned path $P = (p_0, \ldots, p_L)$ with $p_0$ the agent and $p_L$ the goal, samples up to $C = 12$ cells evenly spaced among $p_2, \ldots, p_{L-1}$, and, for each candidate $c$, runs its own A* on the grid with $c$ blocked. It blocks

$$c^* = \arg\max_{c \in C} d_{G \setminus \{c\}}(p_0, p_L), \tag{3}$$

  over candidates that keep the goal reachable, breaking ties toward the cell closest to the agent along the path.

Because the saboteur reads each planner's *own* path, different planners can face different block sequences, as a real opponent who watches the character would produce. We report path length so that this is visible.

**Algorithm 1** One run (one trip of the character)

```
copy the map; a ← start; INIT(grid, start, goal)
steps ← 0; maxSteps ← 10 · opt + 100
while a ≠ goal and steps < maxSteps do
    a ← NEXT_STEP; steps ← steps + 1
    if steps mod K = 0 and blocks < B then
        X ← prankster.ACT(grid, a, goal, CURRENT_PATH())
        if X ≠ ∅ then
            block X; ON_CHANGE(a, X) ▷ timed, logged
        else
            ON_MOVE(a)
        end if
    else
        ON_MOVE(a)
    end if
end while
```

### D. One run

Algorithm 1 shows one run. The pseudo-code of our project plan called neither `on_change` nor `on_move` when a prankster acted but blocked nothing; D* Lite then planned from a stale position and took 35 steps on a 29-step corridor. We therefore call `on_move` in that case; a test now guards this.

## V. Experimental Setup

### A. Maps and problems

We use three sets of the Moving AI 2D grid benchmarks [14]: *dao* (maps from Dragon Age: Origins), *street* (city maps) and *room* (rooms connected by doorways). Cells marked `.`, `G` and `S` are free; all others, including water, are blocked. The benchmark's scenario files give start and goal pairs together with an *octile* (8-direction) optimal length, which is not valid for our 4-direction movement, so we only take start and goal pairs from them and compute the true length with our own BFS. Per set we chose, with a fixed seed, 8 maps with at least 30% free cells and a side of at most 256 cells, and

TABLE II
DATA. BFS LENGTH = TRUE 4-DIRECTION SHORTEST PATH (MEDIAN OVER CHOSEN PROBLEMS).

| Set | Maps | Usable | Chosen | Problems | BFS length |
|---|---|---|---|---|---|
| dao | 156 | 64 | 8 | 120 | 87 |
| street | 90 | 30 | 8 | 120 | 163 |
| room | 40 | 40 | 8 | 120 | 157 |
| Total | 286 | 134 | 24 | 360 | range 60–250 |

per map 15 problems with a BFS length between 60 and 250 steps (Table II). All room maps are 512×512, so for this set only we allowed sides up to 512. Maps that could not supply 15 such problems (small dao maps whose problems are all shorter than 60) were skipped.

### B. Runs

Every problem was run with every planner and every prankster: $360 \times 4 \times 4 = 5760$ runs, with identical maps, problems, seeds and rules for all planners. A run stops at the goal or after $10 \cdot opt + 100$ steps. The pranksters placed 33,892 (random global), 34,091 (random local) and 38,612 (saboteur) blocks in total. The full run took 2 h 43 min with 11 worker processes on a laptop (Intel Core i5-13420H, Python 3.11).

### C. Measures

We measure, per run, in expansions: the *total effort* (first plan plus all replans), the *replanning effort* (all replans) and the *hitch*, the largest number of expansions in any single replan, which is what a player would notice as a stall. Wall-clock time (`time.perf_counter_ns`) is reported only as a secondary measure, because Python timings are noisy: two identical runs differed in their timing columns by a median of 6–38%.

### D. Statistics

Means are reported with 95% percentile bootstrap confidence intervals [23] (2000 resamples, seeded; all planners in a cell resample the same problems). Planners are compared *in pairs on the same problem* (same map, start, goal and prankster seed): D* Lite against A* lazy and against A* restart, per prankster, overall and per map set. We report the Wilcoxon signed-rank test [24] (zero differences dropped), the counts of problems where D* Lite searched less, the same or more (W/T/L), the median ratio D* Lite/baseline, and the matched-pairs rank-biserial correlation [25]

$$r = \frac{R_{\text{favourable}} - R_{\text{unfavourable}}}{R_{\text{favourable}} + R_{\text{unfavourable}}}, \tag{4}$$

where $R$ are the sums of the ranks of $|d|$ for the differences that favour and do not favour D* Lite ($r > 0$ favours D* Lite). Because we run 80 tests, all $p$ values are Holm adjusted [26]; we call a difference significant if the adjusted $p < 0.05$.

### E. Data handling

52 saboteur runs on room maps hit the step limit; re-run with $30 \cdot opt + 100$ they matched exactly in every count and change event (all blocks happen by step $K \cdot B = 150$), so we merged them, and all 5760 runs reached the goal. Our simulator asks D* Lite's `current_path` for up to 10,000 cells instead of the game's 2000; the longest path we measured was 1999 cells.

## VI. RESULTS

### A. Against random changes, lazy A* has the lowest total effort

Table III and Fig. 1 show the total effort. Without changes, every planner only makes its first plan, and D* Lite's costs more: 2,599 expansions on average against 1,571 for A*, and a median 1.84× A*'s per problem (interquartile range 1.06–4.95; D* Lite was cheaper in 80 of 360 problems). Random blocks rarely hit the planned path (0.5% of random-global and 4.5% of random-local blocks did), so lazy A*

TABLE III

TOTAL SEARCH EFFORT (FIRST PLAN + ALL REPLANS) (ALL MAPS, $n = 360$ RUNS PER CELL). MEAN EXPANSIONS WITH 95% BOOTSTRAP CONFIDENCE INTERVAL.

| Planner | None | Random global | Random local | Saboteur |
|---|---|---|---|---|
| BFS restart | 17,701 [16,042, 19,419] | 266,291 [237,961, 295,000] | 268,998 [239,035, 299,447] | 1,137,497 [986,324, 1,297,714] |
| A* restart | 1,571 [1,387, 1,773] | 18,687 [16,141, 21,407] | 18,874 [16,154, 21,787] | 354,770 [284,301, 429,135] |
| A* lazy | 1,571 [1,387, 1,773] | 1,641 [1,447, 1,846] | 2,328 [1,987, 2,688] | 354,770 [284,301, 429,135] |
| D* Lite | 2,599 [2,375, 2,846] | 2,610 [2,379, 2,841] | 2,718 [2,494, 2,968] | 86,989 [70,247, 105,402] |

TABLE IV

PAIRED COMPARISONS OF D* LITE WITH BOTH A* VARIANTS FOR TOTAL EFFORT AND THE HITCH (ALL MAPS, $n = 360$ PAIRED PROBLEMS). W/T/L = PROBLEMS WHERE D* LITE SEARCHED LESS / THE SAME / MORE; RATIO = MEDIAN D*/BASELINE OVER THE $n$ PROBLEMS WHERE THE BASELINE IS $> 0$; $r$ = MATCHED-PAIRS RANK-BISERIAL CORRELATION (POSITIVE FAVOURS D* LITE); $p$ = WILCOXON SIGNED-RANK, HOLM-ADJUSTED OVER ALL 80 TESTS.

| Metric | Prankster | Comparison | W/T/L | Median ratio ($n$) | $r$ | $p$ (Holm) |
|---|---|---|---|---|---|---|
| Total effort | None | D* Lite vs A* lazy | 80/1/279 | 1.8 (360) | −0.68 | < 0.001 |
| Total effort | None | D* Lite vs A* restart | 80/1/279 | 1.8 (360) | −0.68 | < 0.001 |
| Total effort | Random global | D* Lite vs A* lazy | 84/1/275 | 1.7 (360) | −0.64 | < 0.001 |
| Total effort | Random global | D* Lite vs A* restart | 314/0/46 | 0.22 (360) | +0.92 | < 0.001 |
| Total effort | Random local | D* Lite vs A* lazy | 116/1/243 | 1.6 (360) | −0.37 | < 0.001 |
| Total effort | Random local | D* Lite vs A* restart | 315/0/45 | 0.23 (360) | +0.91 | < 0.001 |
| Total effort | Saboteur | D* Lite vs A* lazy | 305/0/55 | 0.36 (360) | +0.89 | < 0.001 |
| Total effort | Saboteur | D* Lite vs A* restart | 305/0/55 | 0.36 (360) | +0.89 | < 0.001 |
| Worst single replan | Random global | D* Lite vs A* lazy | 35/188/137 | 0.033 (37) | −0.28 | 0.011 |
| Worst single replan | Random global | D* Lite vs A* restart | 356/0/4 | 0 (360) | +1.00 | < 0.001 |
| Worst single replan | Random local | D* Lite vs A* lazy | 207/19/134 | 0.034 (236) | +0.62 | < 0.001 |
| Worst single replan | Random local | D* Lite vs A* restart | 348/0/12 | 0.0058 (360) | +0.96 | < 0.001 |
| Worst single replan | Saboteur | D* Lite vs A* lazy | 213/0/147 | 0.82 (360) | +0.17 | 0.042 |
| Worst single replan | Saboteur | D* Lite vs A* restart | 213/0/147 | 0.82 (360) | +0.17 | 0.042 |

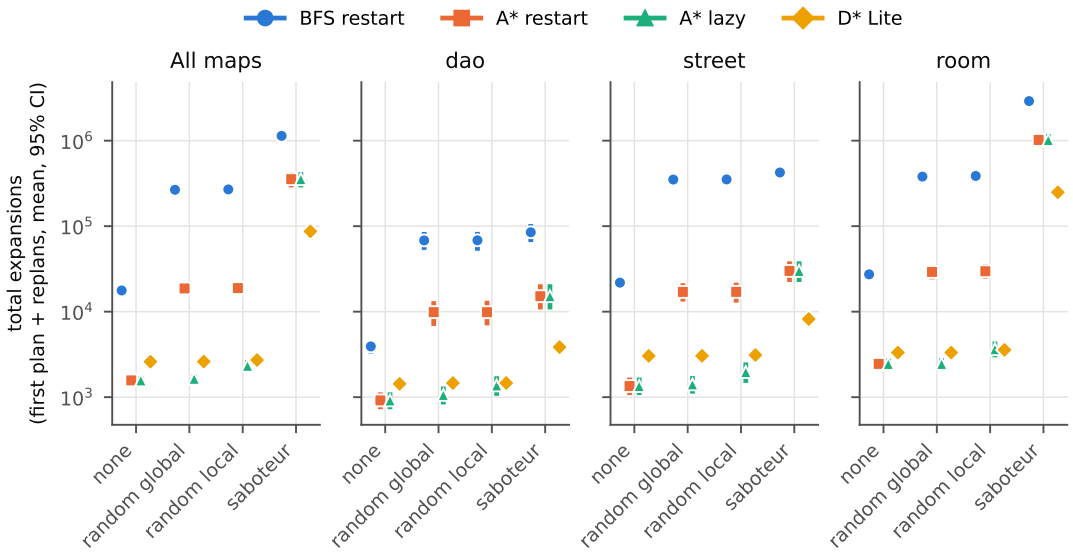



Fig. 1. Total search effort per run (first plan plus all replans), mean with 95% bootstrap CI, log scale, for all maps and per map set ($n = 360$ runs per point for all maps, 120 per set).

hardly ever replans and keeps the lowest mean total effort of all planners (1,641 and 2,328 expansions). D* Lite does a little repair work after most blocks but cannot earn back its more expensive first plan: its total effort is higher than lazy A*'s in 275 (random global) and 243 (random local) of 360 problems (median ratios 1.7 and 1.6, $r = -0.64$ and $-0.37$, both Holm $p < 0.001$; Table IV). Against A* restart, which searches again after every block, D* Lite's total effort is lower (median ratios 0.22 and 0.23, $r = +0.92$ and $+0.91$), and BFS restart is by far the most expensive planner throughout.

### B. Against the saboteur, D* Lite needs about 4× fewer expansions

Every saboteur block lies on the character's path (100% of saboteur blocks), so lazy A* has to replan at every event, exactly like A* restart: under the saboteur the two produced identical runs on all 360 problems. D* Lite's mean total effort is 86,989 expansions against 354,770 for both A* variants, about 4.1× fewer; it searched less in 305 of the 360 problems (median ratio 0.36, $r = +0.89$, Holm $p < 0.001$). For the replanning effort alone the picture is the same and even clearer: 84,390 against 353,199 expansions on average, less in 351 of 360 problems (median ratio 0.27, $r = +0.98$, Holm $p < 0.001$; Fig. 2). The direction holds on every map set (Table V): the median ratio of total effort is 0.41 on dao, 0.54 on street and 0.29 on room. Across all per-set tests, no map set shows a significant result in the opposite direction to the overall one; seven comparisons that are significant overall are not significant in one set (random-global replanning and hitch against A* lazy on dao, the saboteur hitch on street and room, and random-local total effort against A* lazy on room).

TABLE V
D* Lite vs. A* lazy per map set. Total effort: median D* Lite/A* lazy ratio (Holm $p$); hitch: W/T/L and Holm $p$. Columns as in Table IV.

| Set | $n$ | Total, random global | Total, saboteur | Hitch, saboteur W/T/L | $p$ |
|---|---|---|---|---|---|
| Dao | 120 | 1.6 (<0.001) | 0.41 (<0.001) | 85/0/35 | <0.001 |
| Street | 120 | 3.3 (<0.001) | 0.54 (<0.001) | 67/0/53 | 0.351 |
| Room | 120 | 1.4 (<0.001) | 0.29 (<0.001) | 61/0/59 | 1.000 |
| All | 360 | 1.7 (<0.001) | 0.36 (<0.001) | 213/0/147 | 0.042 |

### C. The hitch is not reliably reduced

The worst single replan is shown in Fig. 3 and Table IV. Against the random pranksters, D* Lite's hitches are small and far below A* restart's. Against the saboteur, however, the evidence is weak. The paired test favours D* Lite over both A* variants (213 problems with a smaller hitch, 147 with a larger one; median ratio 0.82, $r = +0.17$, Holm $p = 0.042$), but the effect is small and is not significant on street (Holm $p = 0.351$) or room (61 vs. 59 problems, Holm $p = 1.000$); only on dao is it clear (85 vs. 35, $r = +0.55$). The typical hitch is smaller (median 1,049 against 1,697 expansions), but the mean is the same or slightly larger (20,442 against 20,388): some D* Lite repairs are very large. A player notices exactly these rare, large stalls, so our data do not support the claim that D* Lite avoids hitches against a saboteur.

### D. Where the saboteur blocks matters

Grouping the change events by the distance from the blocked cell to the goal shows that, under the saboteur, D* Lite's advantage depends on that distance. For blocks within 25 cells of the goal it has no advantage at all (1,387 against 1,370 expansions per event for A*), while for blocks 100–149 cells from the goal it needs about 7× fewer (2,505 against 18,441). This matches the structure of the algorithm: D* Lite searches backwards from the

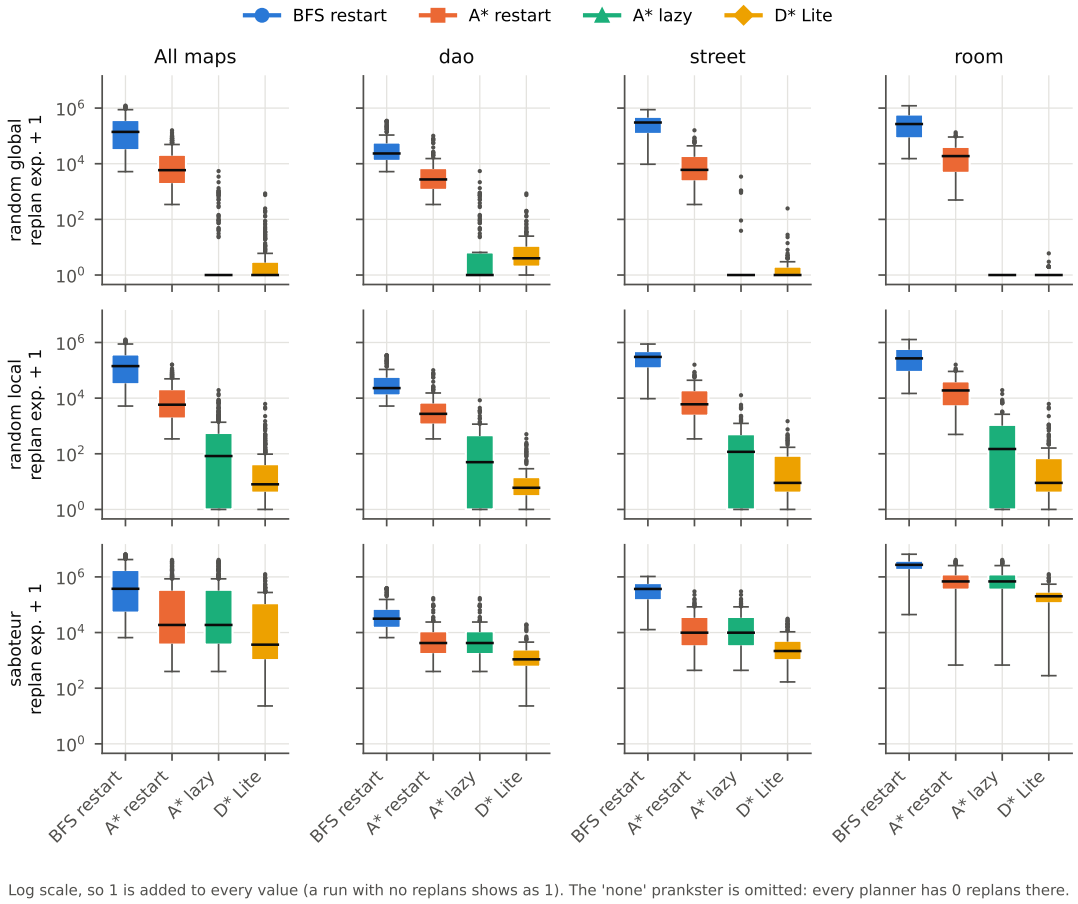



Fig. 2. Replanning effort per run by planner, prankster (rows) and map set (columns). Box: quartiles; line: median; points: outliers. Log scale, so 1 is added to every value (a run without replans shows as 1). Without changes every planner has 0 replans (not shown).

goal, so a block near the goal invalidates a large part of its stored search, while A* restart's cost depends mostly on the distance from the agent to the goal.

### *E. In Python, D* Lite's expansions cost more time*

Table VI reports time as a secondary measure. Measured over all runs, an expansion cost about 54–70 $\mu$s in D* Lite, 16–18 $\mu$s in A* and 3.5–4 $\mu$s in BFS: each D* Lite expansion recomputes *rhs* for its neighbours and updates the queue, while a BFS expansion only appends to a list. This 3–4× difference is far larger than the timing noise (6–38%) and it cancels D* Lite's advantage in expansions: under the saboteur, D* Lite needs about 4× fewer expansions than A* restart, yet its mean time per run is not lower (6,079 against 5,573 ms). That 9% difference itself is within the noise, so we do not claim that D* Lite is slower, only that its expansion savings do not become time savings in this Python implementation. The time per run also leaves out D* Lite's per-step bookkeeping calls, which did no expansions but whose time was not logged.

### *F. Path length*

All planners follow optimal routes on the current map, so their paths differ only through the saboteur's reactions. Without changes and with random changes every planner walks essentially the optimal length (mean ratio 1.0). Against the saboteur the

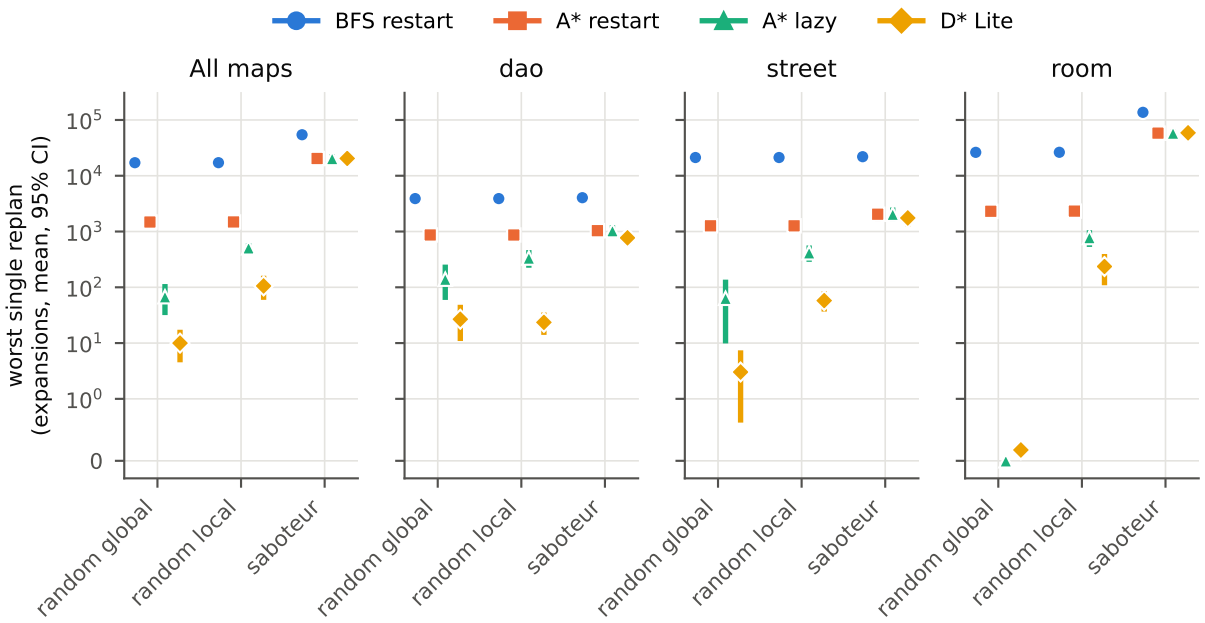



Fig. 3. Worst single replan per run (the "hitch"), mean with 95% bootstrap CI. Symmetric log scale (linear below 1), so a mean of exactly 0 (A* lazy on room against random global blocks never replanned) is still drawn.

TABLE VI

SECONDARY MEASURE: MEAN SEARCH TIME PER RUN IN MS (FIRST PLAN + REPLANS), ALL MAPS, $n = 360$ RUNS PER CELL, 95% BOOTSTRAP CI. NOT USED IN ANY STATISTICAL TEST. TIME PER EXPANSION OVER ALL RUNS ($\mu$S): BFS RESTART: 3.8; A* RESTART: 15.7; A* LAZY: 15.6; D* LITE: 68.6.

| Planner | None | Random global | Random local | Saboteur |
|---|---|---|---|---|
| BFS restart | 65.6 [59.3, 72.4] | 918 [810, 1,033] | 946 [832, 1,059] | 4,429 [3,795, 5,069] |
| A* restart | 28.8 [24.8, 32.9] | 298 [253, 344] | 302 [257, 349] | 5,573 [4,373, 6,814] |
| A* lazy | 27.8 [24.1, 31.8] | 28.7 [24.6, 33.0] | 40.5 [34.6, 47.3] | 5,534 [4,367, 6,737] |
| D* Lite | 140 [128, 153] | 140 [128, 154] | 150 [137, 165] | 6,079 [4,831, 7,449] |

trips become 2.8–2.9× longer on average for every planner, and up to 23.6× on room maps, where closing a doorway forces the character around whole rooms.

## VII. APPLICATION: OUTWIT

Outwit is a Godot game in which the player is a knight guarding a castle and an AI thief tries to steal a treasure chest and escape. The knight locks doors (key E) and drops barrel barricades (key Q) to stop the thief, so in Outwit *the player is the saboteur*: every change the player makes is chosen to block the thief's route. The thief senses the castle through a seven-cell field of view, decides between stealing, fighting and fleeing, and follows its plan with D* Lite, calling `move_start` after every step and `notify_changed` with the cells whose status it has just seen change. The game's own tests show the benefit on its first level: when a wall drops on the route after four steps, D* Lite repairs the path with 3 expansions, while a fresh A* search needs 58, and the repaired path has the same length (24 cells) as the fresh one (Table I).

Our results support this design choice, with limits. Measured in expansions, an opponent who blocks the route on purpose is exactly the case in which D* Lite needs about four times less search

than both A* variants, and in which lazy A* loses its advantage because every block hits its path. D* Lite is therefore the right choice for the thief *in search effort*. Three differences between the game and our experiment limit this conclusion: the thief does not know the whole map but senses it, the knight can also unlock doors and the thief can smash barricades (cells become free again, which our block-only experiment does not test), and GDScript, like Python, is an interpreted language, so the per-expansion cost that cancelled D* Lite's savings in time in our Python port may matter in the game as well. We did not measure the game's frame times; a player would also notice the rare large repairs that D* Lite did not avoid in our data.

## VIII. Discussion and Threats to Validity

### A. Discussion

The answer depends on what the opponent does. Random changes seldom touch the route, so lazy A* wins; changes aimed at the route break it every time, so lazy A* degenerates into A* restart and D* Lite's reuse of its earlier search saves about three quarters of the work. This adds a new axis to the open question of when incremental search pays [9], [10]: whether the changes are aimed at the route.

### B. Threats to validity

- **Timing noise.** Python times depend on load, power state and the interpreter; identical runs differed by 6–38%. We draw no conclusion from time alone, and expansions do not measure the different cost of an expansion in each algorithm.
- **Block-only changes.** Cells are only ever blocked. This is exactly the case in which lazy A* is safe; with cells that become free again, the results may differ.
- **Full knowledge.** The character learns every change at once; our game's thief senses the world through a limited field of view.
- **Grid model.** Four directions, unit costs and the Manhattan heuristic; water is treated as blocked. The results may not carry over to eight-direction movement or weighted terrain.
- **Map selection.** Map sides are at most 256 cells, except for room maps (all $512 \times 512$); 8 maps and 15 problems per set with BFS lengths of 60–250. Because the room maps are larger, map set and map size are confounded.
- **The saboteur reacts to each planner's own path,** so planners do not face identical block sequences, and their trajectories and path lengths differ on the same problem.
- **Tie order.** The port reproduces the game's numbers on its level, but that level cannot distinguish different orders of cells with exactly equal keys; equality with the game's tie order beyond that level is untested.
- **A greedy, one-step saboteur.** A stronger interdictor [12] might find different, possibly more damaging, blocks.

## IX. Conclusion

On 360 problems from real game maps, lazy A* did the least search against random changes, while against a saboteur D* Lite needed about four times fewer expansions than both A* variants. Its worst single replan was not reliably smaller, and in Python its costlier expansions cancelled its savings in time. For a game in which the player is the saboteur, such as Outwit, D* Lite is the better choice in search effort; its repair spikes and per-expansion cost should next be measured in the game itself, together with partial sensing, opponents that also free cells, and stronger interdiction models.

## Acknowledgment

We thank our mentor, Dr. K. Sripriyan, School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, for guidance throughout this project. The maps are from the Moving AI Lab benchmarks [14] (Open Data Commons Attribution License). The code, data and analysis are in our project repository.

## References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[2] J. Orkin, "Three states and a plan: The A.I. of F.E.A.R." in *Proceedings of the Game Developers Conference (GDC)*, San Jose, CA, USA, 2006.

[3] M. Ç. Uludağlı and K. Oğuz, "Non-player character decision-making in computer games," *Artificial Intelligence Review*, vol. 56, no. 12, pp. 14 159–14 191, 2023.

[4] S. Koenig, M. Likhachev, Y. Liu, and D. Furcy, "Incremental heuristic search in artificial intelligence," *AI Magazine*, vol. 25, no. 2, pp. 99–112, 2004.

[5] A. Stentz, "The focussed D* algorithm for real-time replanning," in *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, 1995, pp. 1652–1659.

[6] S. Koenig, M. Likhachev, and D. Furcy, "Lifelong planning A*," *Artificial Intelligence*, vol. 155, no. 1–2, pp. 93–146, 2004.

[7] S. Koenig and M. Likhachev, "D* Lite," in *Proceedings of the 18th National Conference on Artificial Intelligence (AAAI)*, 2002, pp. 476–483.

[8] ——, "Fast replanning for navigation in unknown terrain," *IEEE Transactions on Robotics*, vol. 21, no. 3, pp. 354–363, 2005.

[9] C. Hernández, J. A. Baier, T. Uras, and S. Koenig, "Position paper: Incremental search algorithms considered poorly understood," in *Proceedings of the 5th Annual Symposium on Combinatorial Search (SoCS)*, 2012, pp. 159–161.

[10] C. Hernández, J. A. Baier, and R. Asín, "Making A* run faster than D*-Lite for path-planning in partially known terrain," in *Proceedings of the 24th International Conference on Automated Planning and Scheduling (ICAPS)*, 2014, pp. 504–508.

[11] D. R. Fulkerson and G. C. Harding, "Maximizing the minimum source-sink path subject to a budget constraint," *Mathematical Programming*, vol. 13, no. 1, pp. 116–118, 1977.

[12] E. Israeli and R. K. Wood, "Shortest-path network interdiction," *Networks*, vol. 40, no. 2, pp. 97–111, 2002.

[13] J. C. Smith and Y. Song, "A survey of network interdiction models and algorithms," *European Journal of Operational Research*, vol. 283, no. 3, pp. 797–811, 2020.

[14] N. R. Sturtevant, "Benchmarks for grid-based pathfinding," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 2, pp. 144–148, 2012.

[15] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

[16] A. Stentz, "Optimal and efficient path planning for partially-known environments," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, vol. 4, San Diego, CA, USA, 1994, pp. 3310–3317.

[17] G. Ramalingam and T. Reps, "An incremental algorithm for a generalization of the shortest-path problem," *Journal of Algorithms*, vol. 21, no. 2, pp. 267–305, 1996.

[18] M. Likhachev, D. Ferguson, G. Gordon, A. Stentz, and S. Thrun, "Anytime dynamic A*: An anytime, replanning algorithm," in *Proceedings of the 15th International Conference on Automated Planning and Scheduling (ICAPS)*, 2005, pp. 262–271.

[19] X. Sun, W. Yeoh, and S. Koenig, "Moving target D* Lite," in *Proceedings of the 9th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2010, pp. 67–74.

[20] A. Botea, M. Müller, and J. Schaeffer, "Near optimal hierarchical path-finding," *Journal of Game Development*, vol. 1, no. 1, pp. 7–28, 2004.

[21] D. Harabor and A. Grastien, "Online graph pruning for pathfinding on grid maps," in *Proceedings of the 25th AAAI Conference on Artificial Intelligence*, 2011, pp. 1114–1119.

[22] J. A. Baier, A. Botea, D. Harabor, and C. Hernández, "Fast algorithm for catching a prey quickly in known and partially known game maps," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 7, no. 2, pp. 193–199, 2015.

[23] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[24] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[25] D. S. Kerby, "The simple difference formula: An approach to teaching nonparametric correlation," *Comprehensive Psychology*, vol. 3, 2014.

[26] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.